ACDL 2026 Project: Design and Implementation of a Small Multimodal Sparse Mixture of Experts (MoE) Model for Text, Speech, and Images

1. Project Objective

The objective of this project is to design, implement, train, and evaluate a **small multimodal language model based on a Sparse Mixture of Experts (MoE) architecture** capable of processing and reasoning over: **Text, Speech (audio), Images**
The project must be implemented starting from the open-source **Phi-mini-MoE-instruct** model and extended with dedicated modality encoders and a multimodal fusion mechanism.
The implementation should demonstrate that different experts specialize on different input modalities while sharing a common transformer backbone.
The project is intended to be feasible within **4 weeks**, making extensive use of **Large Language Models (ChatGPT, GitHub Copilot, Claude Code, Gemini, etc.)** as programming assistants.

2. Base Model

Use as starting point: **Phi-mini-MoE-instruct**

https://huggingface.co/microsoft/Phi-mini-MoE-instruct

The student is expected to:

- understand the architecture;
- extend it to multimodal inputs;
- implement and test the complete pipeline.

3. Architecture

The final architecture must contain:

3.1 Text Encoder

Use the tokenizer already provided with Phi-mini-MoE.

Accepted libraries:

- HuggingFace Transformers
- Tokenizers

3.2 Speech Encoder

Replace the video encoder with a speech encoder.

Possible choices include:

- Whisper encoder
- Wav2Vec2
- HuBERT

The speech encoder should transform audio into embeddings compatible with the transformer hidden dimension.

3.3 Vision Encoder

Use one of:

- CLIP Vision Encoder
- ViT
- SigLIP

The encoder should produce image embeddings aligned with the language model embedding space.

3.4 Multimodal Fusion

Implement one of the following:

- Projection layer + concatenation
- Cross-attention
- Adapter-based fusion

The student must explain and justify the chosen design.

3.5 Sparse Mixture of Experts

Replace standard FFN layers with Sparse MoE layers.

Required characteristics:

- Top-2 gating
- SwiGLU experts
- Capacity factor
- Auxiliary load-balancing loss
- bfloat16 or float16 training when supported

The implementation should report:

- routing probabilities;
- expert utilization;
- entropy of gate assignments;
- percentage of inactive experts.

4. Training Datasets

To keep the project computationally feasible, only **small subsets** (or sampled versions) of the datasets should be used.

4.1 Text Dataset

Choose **one**:

- The Pile
- C4

4.2 Programming Dataset

Choose **one**:

- CodeParrot
- The Stack v2

4.3 Logical Reasoning Dataset

Choose **one**:

- LogiQA
- ReClor

4.4 Mathematical Dataset

Choose **one**:

- GSM8K
- MATH

4.5 Educational Reasoning Dataset

Choose **one**:

- SAT
- GRE
- GMAT

4.6 Image Dataset

Choose **one**:

- COCO
- ImageNet
- LAION subset

4.7 Speech Dataset

Choose **one**:

- LibriSpeech
- Mozilla Common Voice
- VoxPopuli
- GigaSpeech (small subset)

The speech data should be converted into embeddings using the selected speech encoder.

5. Data Preprocessing

The preprocessing pipeline must include:

- text tokenization;
- image resizing and normalization;
- audio loading;
- resampling to 16 kHz;
- padding/truncation;
- batching;
- multimodal alignment.

The complete preprocessing code must be reproducible.

6. Training Pipeline

Implement the training pipeline in PyTorch.

Suggested libraries:

- transformers
- datasets
- accelerate
- peft
- deepspeed (optional)

Training should include:

- checkpoint saving;
- evaluation every epoch;
- logging;
- reproducibility through random seeds.

7. Evaluation

Evaluate the model on:

Text

- perplexity
- accuracy

Images

- image-text retrieval or caption matching accuracy

Speech

- speech-text retrieval or transcription embedding similarity

MoE Analysis

Produce plots showing:

- expert load distribution;
- gate entropy;
- routing frequency;
- token allocation per expert.

8. Repository Structure

The final GitHub repository must contain:

```
model/
  modeling_moe.py
  multimodal_model.py

datasets/
  preprocess_text.py
  preprocess_images.py
  preprocess_audio.py

training/
  train.py
  config.yaml

evaluation/
  evaluate.py
  metrics.py

notebooks/
  demo.ipynb

README.md
requirements.txt
run.sh
```

The repository **must execute without modifications** after:

```
pip install -r requirements.txt
bash run.sh
```

The GitHub repository must be public and contain:

- source code;
- documentation;
- installation instructions;
- inference example;
- training instructions.

9. Technical Report

Submit a report (minimum **15 pages**) including:

1. Introduction
2. Background on Sparse Mixture of Experts
3. Background on Multimodal Large Language Models
4. Description of the datasets
5. Architecture
6. Implementation details
7. Training methodology
8. Experimental evaluation
9. Expert routing analysis
10. Discussion
11. Conclusions

Figures should include:

- architecture diagram;
- training loss;
- expert utilization histogram;
- routing entropy;
- evaluation results.

10. Deliverables

The student must submit:

A.

A public GitHub repository containing the **fully functional source code**.

The code must execute successfully and reproduce the reported experiments.

B.

The final technical report in PDF format.

---

11. Recommended Computational Resources

The project is designed to be completed without access to expensive computing infrastructure.

Suitable platforms include:

Option 1 (recommended)

- Google Colab Free
- Google Colab Pro

using a T4, L4, or A100 GPU when available.

Option 2

A personal laptop equipped with:

- NVIDIA GPU with at least 8 GB VRAM

or

- Apple Silicon (M1/M2/M3) with at least 16 GB unified memory.

Option 3

Free academic computing resources such as:

- Kaggle Notebooks
- Lightning AI Studio (free tier)
- university HPC clusters
- EuroHPC training allocations
- institutional GPU servers

Training on reduced dataset subsets is acceptable and encouraged.

12. Suggested Four-Week Schedule

Week 1

- environment setup;
- study Phi-mini-MoE;
- dataset download;
- preprocessing implementation.

Week 2

- implement image encoder;
- implement speech encoder;
- implement multimodal fusion.

Week 3

- implement Sparse MoE routing;
- perform training;
- debug the pipeline.

Week 4

- evaluation;
- generate plots;
- prepare GitHub repository;
- write technical report;
- verify reproducibility.

The project should emphasize code quality, reproducibility, and a clear understanding of Sparse Mixture of Experts for multimodal artificial intelligence.